Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering

Assignment 2

ELC 4028 – Artificial Neural Networks

ID	Section	Name
9220399	2	Shahd Gamal Mahmoud Hussein
9220411	2	Salah El-Din Hassan Hassan Mohamed
9220984	4	Yousef Khaled Omar Mahmoud

Supervised by:

Dr. Mohsen Rashwan
Dr. Mohamed Motaz

Contents

Problem 1 **3**

1 Autoencoder / MLP Analysis **3**
1.1 Dataset and Preprocessing . . . 3
1.2 Model Architecture and Decisions . . . 3
1.2.1 Mathematical Formulation . . . 3
1.3 Training Procedure . . . 3
1.4 Results . . . 4
1.5 Discussion and Decisions . . . 4
1.6 Conclusion . . . 4

Problem 2 **5**

2 CNN Experiments (LeNet-style) **5**
2.1 Design Decisions . . . 5
2.2 Mathematical and Implementation Details . . . 5
2.3 Experimental Protocol . . . 5
2.4 Results and Findings . . . 5
2.5 Discussion . . . 6
2.6 Conclusion and Recommendations . . . 6

Problem 3 **7**

3 Placeholder **7**
3.1 Results . . . 7

Problem 4 **8**

4 Placeholder **8**
4.1 Results . . . 8

Problem 5 **9**

5 Placeholder **9**
5.1 Results . . . 9

Problem 6 **10**

6 Placeholder **10**
6.1 Results . . . 10

Conclusion **11**

List of Figures

1 Training loss curve (Problem 1). 4
2 Per-epoch train/test accuracy curves for LeNet variants. 6

Problem 1

1 Autoencoder / MLP Analysis

1.1 Dataset and Preprocessing

The dataset used, directory layout, and preprocessing steps are listed below:

- Grayscale input conversion to single channel
- Rescaling / normalization: placeholder for mean/std (to be filled)
- Train / test split as provided in the assignment dataset

1.2 Model Architecture and Decisions

We implemented an MLP / autoencoder-style architecture. Key design decisions:

- Hidden layer sizes and activation functions selected to balance capacity and training stability.
- Loss: cross-entropy for classification tasks or mean-squared error for reconstruction tasks, denoted here as $\mathcal{L}$.

1.2.1 Mathematical Formulation

For classification we use softmax cross-entropy. Given logits $z \in \mathbb{R}^C$ for C classes, the predicted probabilities are

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}},$$

and the loss for a single example with one-hot target y is

$$\mathcal{L}_{CE} = -\sum_{c=1}^{C} y_c \log p_c.$$

For reconstruction tasks with predicted output $\hat{x}$ and target x, we use MSE:

$$\mathcal{L}_{MSE} = \|x - \hat{x}\|_2^2.$$

1.3 Training Procedure

Hyperparameters used (placeholders; fill with actual values from runs):

- Optimizer: SGD/Adam
- Learning rate: 0.001 (example)
- Batch size: 128
- Epochs: 5

Training and evaluation routines measured wall-clock time using high-precision timers; reported metrics include training accuracy, test accuracy, training time, and test time.

Table 1: Problem 1: Summary of results

Model	Train Acc (%)	Test Acc (%)	Train Time (s)
Baseline MLP	–	–	–
Variant A	–	–	–

Placeholder: prob1_train_loss.png

Figure 1: Training loss curve (Problem 1).

1.4 Results

Insert accuracy and timing tables and plots here. Example table template:

1.5 Discussion and Decisions

Summarize observed behaviour, issues encountered (e.g., GPU compatibility), and rationale for chosen hyperparameters.

1.6 Conclusion

State final conclusions and next steps.

Problem 2

2 CNN Experiments (LeNet-style)

2.1 Design Decisions

We implemented a baseline `LeNet5` with the following structure:

- Convolutional layer 1: 5×5, 6 filters, followed by average pooling (2x2)
- Convolutional layer 2: 5×5, 16 filters, followed by average pooling (2x2)
- Fully connected layers: flatten $\rightarrow$ 120 $\rightarrow$ 84 $\rightarrow$ 10 (logits)

Variants added to investigate capacity and regularisation effects:

Wide: increase conv filters to (16,32) to raise model capacity.

Deep: add an extra 3×3 conv layer (padding=1) increasing depth and receptive field.

Sigmoid: replace ReLU activations in the convolutional and FC layers with Sigmoid activations (no dropout).

Each variant is constructed by a factory function and run with the same training pipeline to ensure fair comparison.

2.2 Mathematical and Implementation Details

Convolutional operations are denoted as

$$y_k = f\left(\sum_c x_c * w_{c,k} + b_k\right),$$

where $*$ is 2D convolution and $f(\cdot)$ is the activation function (ReLU or Sigmoid for the `sigmoid` variant).

Training minimises cross-entropy loss over the dataset. We measure per-epoch wall-clock training time t_{epoch} and report total training time $T = \sum_e t_{\text{epoch}}^{(e)}$.

2.3 Experimental Protocol

- Data preprocessing: grayscale conversion and tensor transform (placeholders for normalization values).
- Optimizer: Adam with learning rate 0.001 (default; configurable).
- Batch size: 128; Epochs: 5 (default; configurable).
- Metrics: train accuracy, test accuracy, training wall time, testing wall time.

2.4 Results and Findings

The comparative results for the four variants are saved to a CSV (`lenet_variants_results.csv`) and should be inserted as a table here. Example placeholder table:

Table 2: LeNet variants: accuracy and timing summary

Variant	Train Acc (%)	Test Acc (%)	Train Time (s)	Test Time (s)
Baseline	–	–	–	–
Wide	–	–	–	–
Deep	–	–	–	–
Sigmoid	–	–	–	–

Placeholder: prob2_accuracy_curves.png

Figure 2: Per-epoch train/test accuracy curves for LeNet variants.

2.5 Discussion

Discuss why the wide or deep variants behaved differently from baseline, how Sigmoid activation affected training dynamics, and any regularisation or optimisation observations (e.g., learning rate sensitivity).

2.6 Conclusion and Recommendations

Summarise best-performing variant, trade-offs between accuracy and compute (training time), and recommendations for further experiments (longer training, data augmentation, normalization).

Problem 3

3 Placeholder

3.1 Results

Place figures and tables for Problem 3 in this file.

Problem 4

4 Placeholder

4.1 Results

Place figures and tables for Problem 4 in this file.

Problem 5

5 Placeholder

5.1 Results

Place figures and tables for Problem 5 in this file.

Problem 6

6 Placeholder

6.1 Results

Place figures and tables for Problem 6 in this file.

Conclusion

Summarize key findings across all problems, reflect on the learning outcomes, and suggest potential future directions or improvements for the experiments conducted.